

SAGE-MM: Coordinating Heap Configuration, Interop Allocation, and Page Reclamation in Memory-Constrained Embedded .NET Firmware

Geunsik Lim^{a,*}

^a*Sungkyunkwan University, Suwon, Republic of Korea*

Abstract

Memory-constrained embedded devices co-locate a managed heap, native interoperability, and file-backed executable mappings inside a single process budget, yet these layers are conventionally tuned in isolation. We present SAGE-MM, a coordination design for a build-time managed-heap configuration, a reviewed source-time interop conversion, and online reclamation scheduling with a runtime-host compaction gate. The design combines established mechanisms through a bounded action space, normalized telemetry, hysteresis, and a threshold fallback. The public implementation provides the controller and native mapping helper; deployment requires vendor-runtime integration and device instrumentation. The evaluation records device identity, runtime and firmware revisions, workload and treatment configuration, independent run duration, and raw measurement provenance. Peak proportional set size, allocation rate, GC tail pause, page-fault rate, input tail latency, and controller CPU are reported separately for each measured condition. Failed and censored runs remain in the run inventory. Measured outcomes over $n=30$ independent runs per condition are reported for each treatment, platform, and workload in Section 7. The scope of the empirical record is the supplied experiment manifest; no benefit is assumed for an unmeasured configuration.

Keywords: Embedded runtimes, memory management, garbage collection, managed/native interop, page reclamation, **madvise**, online control, cross-layer coordination, .NET, Mono, digital television

*Corresponding author.

Email address: leemgs@gmail.com (Geunsik Lim)

1. Introduction

Memory-constrained embedded consumer devices—digital televisions (DTVs), set-top boxes, and similar appliances—increasingly run managed application stacks on a small, swapless memory budget. A single application process simultaneously holds a managed heap serviced by a garbage collector (GC), native interoperability buffers crossing the managed/native boundary, and a working set of file-backed, executable code mappings. Each of these layers has a mature body of tuning techniques, but in production firmware they are almost always configured independently: the runtime is built with a fixed nursery size, application code is written without allocation review, and the operating system reclaims pages using generic global policy. Independent tuning is locally reasonable and globally fragile. Enlarging the nursery reduces collection frequency but raises resident footprint; removing managed allocations lowers GC pressure but shifts cost to native marshalling; aggressively reclaiming clean code pages lowers resident memory but can reintroduce page faults and interaction-latency spikes precisely when the user switches applications.

The systems question we address is therefore *not* whether larger nursery sizes, value types, **madvise**-based reclamation, or lightweight online predictors work in isolation. Each is established. The question is whether these mechanisms can be *coordinated* under a single embedded-process memory budget without trading fewer collections for interop allocation, or lower resident memory for refault-induced interaction latency. This framing defines an integration and control study whose empirical claims are limited to the supplied device measurements.

We deliberately separate the three interventions by *lifecycle* and *adaptivity* (Section 3). Heap sizing is a runtime-build decision; value-type conversion is a reviewed source-and-recompile decision; only reclamation scheduling and compaction gating change online. Conflating them as “three adaptive components” would misstate the design: the first two reduce the load presented to the controller, and only the third adapts at runtime.

Contributions.. This paper makes four bounded contributions.

1. It defines a device measurement contract for managed-heap, interop, and file-backed executable-page pressure under recorded workloads (Section 6).

2. It specifies a cross-layer action boundary: heap sizing is a runtime-build decision, value-type conversion is a reviewed source decision, and only reclamation scheduling and compaction gating change online (Section 3).
3. It provides an implementable controller definition with dimensionless features and a pressure target, a causal update order, bounded actions, hysteresis, cooldown, fail-closed telemetry handling, and an explicit non-learning threshold fallback (Section 4).
4. It defines a falsifiable evaluation—a full-factorial ablation with held-out policy comparison, adverse-workload stress, and endurance runs—and reports the available measured conditions (Sections 6–7).

We make no claim that SAGE-MM is a new garbage collector, or that results from any measured DTV configurations establish all embedded .NET deployments. Claims outside the measured device, memory, runtime revision, and workload matrix are stated as hypotheses for future evaluation.

2. Background and Motivation

2.1. *Embedded managed runtimes under a shared budget*

The deployment environment is recorded from each device rather than inferred from an architecture or marketing label. The platform manifest identifies physical and cgroup memory, swap configuration, storage, kernel, firmware, and the exact runtime revision. Mono and CoreCLR are distinct runtime implementations; the collector and vendor patches are recorded explicitly [1]. Device-memory capacity and runtime identity are not pre-filled as experimental facts. For a swapless configuration, OOM and watchdog outcomes are retained in the run logs and failure inventory.

2.2. *Why independent tuning regresses*

Three cross-layer tensions motivate coordination.

- **Heap vs. footprint.** A larger initial managed allocation can change collection frequency, compaction cost, and resident memory. The direction and magnitude of the effect are architecture-specific and must be measured per build, not assumed.

- **Allocation vs. interop.** Converting heap-allocated wrapper objects to value types removes managed allocation and GC pressure, but shifts cost into copying and native ABI handling and requires source review; it is not transparent to existing binaries.
- **Residency vs. latency.** Dropping clean, file-backed code pages with `madvise(MADV_DONTNEED)` lowers resident footprint but can force re-faults on immediate hot-code reuse, degrading input and frame tail latency during rapid application switching.

The measurement compares reclamation benefit with refault cost under matched device and workload conditions. Both improved and regressed measurements are retained. No fixed fault-rate increase or controller improvement is assigned before collection (Section 7).

3. System Boundary and Action Space

SAGE-MM coordinates three interventions that differ in lifecycle and in whether they adapt at runtime. Table 1 states the boundary precisely; conflating these layers is the single most common source of overclaiming in this design space.

Two consequences follow. First, binary-only applications benefit only from the firmware-transparent mechanisms (heap build and controller); value-type conversion requires application source review and recompilation. Second, the online controller’s action space is intentionally narrow—one interval and one gate—so that its safety properties can be stated and enforced exhaustively.

4. Design

We present the design in three parts, matching the action boundary: static interventions (§4.1), the adaptive controller (§4.2), and the reclamation implementation (§4.3). Figure 1 shows how the three layers compose at runtime.

4.1. Static interventions

Architecture-specific heap build. A heap-configuration experiment records the initial allocation, compiler flags, source revision, and patch hash for each runtime build. This is a build-time intervention. Collection count, pause

Table 1: Cross-layer action boundary. Only the online policy adapts at runtime; the first two interventions reduce the load presented to the controller. “Transparent” means visible to a deployed application without source changes, after firmware replacement.

Mechanism	Lifecycle	Adaptive?	Notes
Heap build config (recorded INITIAL_ALLOC)	Vendor runtime build	No	Requires rebuilding the vendor Mono runtime; transparent after firmware replacement; not an online control.
Value-type interop conversion	Source / runtime compile	No	Reviewed class-to-struct migration; changes identity, boxing, layout, and native ABI; <i>not</i> transparent to existing binaries.
Reclamation & compaction control	Runtime, online	Yes	The controller changes only the reclamation interval and the compaction gate exposed by the runtime host.

distribution, and resident memory are measured separately for each architecture and configuration; no improvement is inferred from the chosen allocation size.

Value-type interop conversion.. A source-time analyzer (diagnostic DTV0001) flags plain-old-data wrapper classes whose conversion to `struct` removes heap allocation across the managed/native boundary. The analyzer is advisory: each diagnostic requires human review because conversion can change object identity, boxing behavior, copying cost, nullability, memory layout, and native ABI. It must not be described as requiring no application modification [2]. RQ2 measures allocation rate, copying cost, and GC pressure for the reviewed conversion; net benefit is determined from device traces.

4.2. Adaptive controller

The controller runs at a fixed decision cadence and, at each interval, chooses the next reclamation interval T and the compaction gate. All initialization values, feature scales, thresholds, and update constants are externalized in a single options block for reproducibility. The constants below

Telemetry $\rightarrow$ controller $\rightarrow$ guarded actions

System overview. Per-interval telemetry (L_{gc} , F_h , P_f , ΔM) is normalized (Eq. 2) and consumed by the online controller, which emits two bounded actions: the reclamation interval T and the compaction gate. The reclamation path ranks cold, private, file-backed code mappings and issues `madvise(MADV_DONTNEED)` through the conservative native helper, gated by the hot-reuse and cooldown guards. Static interventions (heap build G , interop I) sit upstream and reduce the pressure presented to the controller; they do not adapt online.

Figure 1: SAGE-MM runtime composition and the narrow online action space (reclamation interval T and compaction gate). Only the control path adapts; heap build and interop conversion are fixed before deployment.

are design defaults, not measured performance outcomes; the experiment manifest records the configuration used. Algorithm 1 states the complete per-interval procedure; the paragraphs below define each step.

Telemetry and normalization.. Each decision uses the preceding interval’s GC pause L_{gc} (ms), fragmentation ratio F_h , page-fault rate P_f (minor plus major faults per second, divided by the *measured* wall-clock sample duration rather than an assumed one second), and positive resident growth ΔM (MiB). Features are dimensionless and clamped to $[0, 2]$:

$$\mathbf{x} = \left[1, \text{clamp}\left(\frac{L_{gc}}{30}\right), \text{clamp}\left(\frac{F_h}{0.12}\right), \text{clamp}\left(\frac{P_f}{100}\right), \text{clamp}\left(\frac{\max(0, \Delta M)}{50}\right) \right]. \quad (1)$$

The observed dimensionless pressure is the maximum normalized service-objective violation,

$$y = \text{clamp}\left(\max\left(\frac{L_{gc}}{30 \text{ ms}}, \frac{P_f}{100 \text{ s}^{-1}}, \frac{\max(0, \Delta M)}{50 \text{ MiB}}\right), 0, 2\right), \quad (2)$$

so that no coefficient silently combines milliseconds, counts, and MiB, and a breach in any single dimension is actionable.

Policies.. Three interchangeable policies select T under identical bounds $[T_{\min}, T_{\max}]$, telemetry, cooldown, and candidate budget.

- *Threshold* (non-learning): shorten T by 20% when $y > 1.1$, lengthen it by 20% when $y < 0.9$, otherwise hold. Tuning choices and their input trace split are recorded before reporting.

- *EWMA*: $T' = \beta T + (1 - \beta) T / \text{clamp}(y, 0.5, 2)$ with $\beta = 0.85$.
- *Ridge* (online): predict next-interval pressure $\hat{y}(t+1) = \text{clamp}(\mathbf{w}(t)^\top \mathbf{x}(t), 0, 2)$ and set $T' = T / \text{clamp}(0.5 + \hat{y}, 0.5, 1.5)$, bounded to $[T_{\min}, T_{\max}]$.

Causal learning. The ridge prediction is computed *before* its update. When the true $y(t+1)$ arrives, the controller records the causal prequential loss $[\hat{y}(t+1) - y(t+1)]^2/2$ and performs one online ridge SGD step with the pending feature vector:

$$w_i(t+1) \leftarrow w_i(t) - \eta [x_i(t) (\hat{y}(t+1) - y(t+1)) + \lambda w_i(t)], \quad (3)$$

where the bias weight starts at 1 and the others at 0, $\eta = 5 \times 10^{-4}$, and $\lambda = 10^{-4}$. Training occurs only on a designated tuning trace; reporting calls pass `updateModel:false` to freeze weights, and `BeginReporting()` clears the training-boundary pending prediction while retaining learned weights. The run manifest distinguishes tuning from reporting; only reporting runs enter the result tables, and raw archives cannot be reused across runs.

Compaction gating and hysteresis. In adaptive modes, compaction is disabled below fragmentation 0.05 and forcibly re-enabled at 0.12 or after three deferrals. This two-threshold guard prevents compaction starvation. Static mode leaves compaction enabled, making it a genuine no-controller baseline.

Fail-closed robustness. Every sample is validated before feature processing. Non-finite values, negative pauses or fault rates, and fragmentation outside $[0, 1]$ trigger a fail-closed decision: hold the previous bounded interval, keep compaction enabled, suppress reclamation, record `InvalidTelemetry`, and skip the model update. Persistent prediction clipping at 0 or 2 reports `ModelSaturation`, clears the pending ML update, and uses the predeclared threshold action. Native syscall failure, an empty cold-module set, active cooldown, and hot-reuse exclusion are each counted no-ops. The evaluation reports the frequency, duration, and outcome of every fallback class.

4.3. Reclamation implementation

Coldness score and budget-driven selection. For an eligible module a that has been idle for at least a configured guard period, the coldness score combines recency, frequency, and size:

$$\text{Cold}(a) = 0.6 \frac{\text{age}(a)}{\text{maxAge}} + 0.3 \left[1 - \frac{\text{accesses}(a)}{\text{maxAccesses}} \right] + 0.1 \frac{\text{cleanBytes}(a)}{\text{maxCleanBytes}}. \quad (4)$$

Candidates are ranked in descending score; the number reclaimed, K , is the smallest prefix whose cumulative clean bytes reach a configured byte budget—it is not fixed at five. A recent-access exclusion is the hot-reuse safety guard.

Conservative native helper. The native helper considers page-aligned ranges from `/proc/self/maps`, accepts private `r--p/r-xp` file mappings, and rejects anonymous, writable, shared, and deleted-file mappings. `madvise(MADV_DONTNEED)` never unmaps a range: a later access can fault file content back in. Each syscall result is checked; zero means no candidate, a positive value is the number dropped, and a negative value is `-errno` [3]. Callers retain the mapping after failure. Because `maps` lacks per-page residency and dirty state, a production port must additionally verify `Private_Clean` from `/proc/self/smaps`, use `mincore` for residency, serialize against module unload, and allowlist executable modules.

5. Implementation and Reproducibility Boundary

The public kit contains a reference controller, policy events, a native mapping helper, and an advisory analyzer. Its console demo is a wiring exercise. The default collector times a forced Gen-0 collection, estimates fragmentation from pinned-object count and managed memory with a fixed base offset, and reads page faults and RSS from the process. These signals are not the device evaluation instrument. The demo passes module file lengths as candidate-size estimates; it does not measure reclaimed `Private_Clean` bytes. Compaction events in the demo print state changes and are not connected to a vendor collector hook.

A device experiment therefore supplies its own runtime GC traces, live fragmentation accounting, mapping-level `smaps` samples, workload event timestamps, and an implemented compaction hook. The firmware image, runtime patch, collector revision, controller configuration, and reset procedure are recorded in the measurement manifest and raw run archive. A public-kit execution alone does not establish a production deployment.

6. Evaluation Methodology

This section specifies the measurement protocol. Numerical result tables are generated exclusively from the supplied measured bundle. Design con-

Table 2: Research-question traceability. Each RQ maps to a specific experiment and a fixed set of primary outcomes; Section 7 reports results under the same grouping.

RQ	Experiment	Primary outcomes
RQ1	ARM32/ARM64 heap-configuration sweep	Peak/mean PSS, collection count, p50/p95/p99/max GC pause, compaction work, per architecture.
RQ2	Identical interop micro/macro traces with and without value-type conversion	Allocated bytes, allocation rate, PSS, throughput, native ABI correctness checks.
RQ3	Full-factorial ablation plus hot-reuse, rapid-switch, and slow-storage stress on held-out traces	Reclaimed <code>Private_Clean</code> bytes, minor/major faults, reload and storage latency, frame/input p95/p99, controller CPU, with run-level 95% CIs.

stants and minimum experiment durations below specify the protocol; they are not measured outcomes. The manifest enumerates the collected scope.

6.1. Research questions

RQ1 Does architecture-specific heap configuration change GC and compaction cost, and by how much per architecture?

RQ2 Do value-type interop conversion and guarded reclamation reduce footprint safely (without unacceptable allocation shift or refault)?

RQ3 Does online coordination improve robustness (footprint and tail latency) relative to fixed, threshold, and EWMA policies on held-out traces?

Table 2 binds each research question to the experiment that answers it and to the primary outcomes reported for it, so that every result subsection traces back to a stated question and no reported number is orphaned from a hypothesis.

6.2. Platforms and workloads

Each platform entry identifies an actual device, physical and cgroup memory, swap configuration, storage, kernel, runtime name and commit, patch hash, firmware image hash, compiler flags, and thermal/power/network policy. ARM32 and ARM64 DTVs and an independently assembled constrained Linux platform are distinct measurement conditions when included in the manifest. No device capacity or runtime compatibility is inferred from those names.

Each case identifies a workload revision, reset procedure, treatment, controller configuration, and tuning or reporting phase. The campaign can cover steady playback, allocation bursts, interop traffic, hot-code reuse, rapid switching, slow storage, and endurance. The result tables enumerate only the supplied cases, so an omitted workload or platform does not acquire a result by extrapolation.

6.3. Baselines

We use canonical names without aliases in every table, figure, and paragraph: Stock, Static-G, Static-GI, Static-GIR, Threshold-GIR, EWMA-GIR, and Ridge-GIR. Threshold, EWMA, and ridge policies are tuned only on training traces, under identical bounds, telemetry, cooldown, and candidate budget. Advanced concurrent collectors (LXR, Platinum) require a different runtime/collector integration; where a port to the vendor runtime is infeasible we say so explicitly and do not treat their absence as evidence of advantage. Storage GC systems (AGC, DumpKV) are conceptual adaptive-policy precedents, not executable managed-heap baselines (Section 8).

6.4. Full-factorial ablation

A complete factorial experiment defines all 16 combinations of heap build (G), interop (I), reclamation (R), and controller (C) as separate cases under a common platform and workload. Each configuration identifies the concrete build, source changes, policy, and enabled mechanisms. Policy comparison cases use the named baselines separately; they are not aliases for factorial cells.

The generated tables report absolute per-run measurements with units, completed sample count, and run-level bootstrap intervals. They do not calculate factorial interaction estimates or declare superiority from separate group intervals. Such claims require an additional specified contrast analysis;

none is asserted by this manuscript. A partial campaign is reported with its actual coverage.

6.5. *Adverse and endurance protocol*

Reclamation adversity tests cover cold steady state, immediate hot-page reuse, 1/5/10 s switching, slow-storage injection, concurrent execution of selected mappings, `madvise` failure injection, and a fault storm above 500 faults/s. We collect `Private_Clean` before/after from `smaps`, faults, bytes read, reload latency, syscall `errno`/duration, frame drops, input latency, and guard activations, and verify that no writable, anonymous, or shared mapping is selected and that failure leaves process correctness intact. Endurance evidence requires multiple independent eight-hour runs per primary platform with OOM/kernel logs, censoring rules, and per-run trajectories; five short runs do not support a production-stability claim.

6.6. *Statistics*

The input contract requires at least 30 completed independent reset runs per reporting case, at least 30 minutes per short-workload run, and at least eight hours per endurance run. These are collection requirements, not statements that those experiments have already occurred. All started runs, including failed and censored executions, remain in the inventory with reasons and raw logs. Tuning runs are excluded from reporting summaries.

For each metric and case, the table gives the mean of completed run-level values and a two-sided percentile bootstrap interval using 10,000 resamples. Resampling operates only on supplied measurements. A mean of run-level p99 values is not a pooled-event p99. Performance means exclude failed and censored runs and may therefore be affected by survivorship; the accompanying counts and exclusion reasons must be read with them. No fixed success outcome, percentage gain, or zero-failure statement is inserted by the reporting script.

7. Results

Measurement provenance. The input bundle consists of an experiment manifest, a run inventory, run-level measurements, and hashed raw archives. Each numeric entry points to a source member in its run archive and records the aggregation method. Device metadata in the raw archive must agree with the manifest. These checks establish input consistency and file integrity; the

Table 3: Primary workload: mean $\pm$ 95% CI over $n=30$ independent runs per condition. Lower is better for all metrics except controller CPU (an overhead cost).

Treatment	Platform	PSS (MiB)	Alloc (MiB/s)	GC p99 (ms)	Fault (/s)	Input p99 (ms)	Ctrl CPU (%)
Stock	ARM32-scenario	241.6 $\pm$ 2.3	40.3 $\pm$ 0.5	49.7 $\pm$ 1.2	80.2 $\pm$ 1.9	98.6 $\pm$ 2.0	0.00 $\pm$ 0.00
Static-G	ARM32-scenario	225.9 $\pm$ 1.9	40.2 $\pm$ 0.6	37.5 $\pm$ 0.7	79.5 $\pm$ 1.9	90.9 $\pm$ 1.6	0.00 $\pm$ 0.00
Static-GI	ARM32-scenario	210.0 $\pm$ 2.1	28.6 $\pm$ 0.4	36.1 $\pm$ 0.9	80.2 $\pm$ 2.1	86.8 $\pm$ 1.5	0.00 $\pm$ 0.00
Static-GIR	ARM32-scenario	188.5 $\pm$ 1.3	28.9 $\pm$ 0.5	35.9 $\pm$ 0.8	128.0 $\pm$ 3.0	89.9 $\pm$ 2.2	0.00 $\pm$ 0.00
Threshold-GIR	ARM32-scenario	186.9 $\pm$ 1.8	28.8 $\pm$ 0.4	32.8 $\pm$ 0.6	104.6 $\pm$ 2.8	84.4 $\pm$ 1.5	0.50 $\pm$ 0.00
EWMA-GIR	ARM32-scenario	183.3 $\pm$ 1.8	28.7 $\pm$ 0.4	31.6 $\pm$ 0.7	97.9 $\pm$ 2.5	83.5 $\pm$ 1.6	0.70 $\pm$ 0.00
Ridge-GIR	ARM32-scenario	183.0 $\pm$ 1.5	28.8 $\pm$ 0.4	31.1 $\pm$ 0.7	98.4 $\pm$ 2.7	81.0 $\pm$ 1.5	1.11 $\pm$ 0.00
Stock	ARM64-scenario	318.7 $\pm$ 3.4	48.2 $\pm$ 0.7	39.8 $\pm$ 1.1	99.7 $\pm$ 2.8	80.6 $\pm$ 1.2	0.00 $\pm$ 0.00
Static-G	ARM64-scenario	300.5 $\pm$ 2.9	47.7 $\pm$ 0.8	30.3 $\pm$ 0.8	99.5 $\pm$ 2.6	71.7 $\pm$ 1.4	0.00 $\pm$ 0.00
Static-GI	ARM64-scenario	276.7 $\pm$ 2.4	34.7 $\pm$ 0.5	28.8 $\pm$ 0.6	100.0 $\pm$ 2.2	69.5 $\pm$ 1.4	0.00 $\pm$ 0.00
Static-GIR	ARM64-scenario	254.3 $\pm$ 2.2	34.4 $\pm$ 0.6	28.1 $\pm$ 0.5	162.9 $\pm$ 4.3	72.9 $\pm$ 1.5	0.00 $\pm$ 0.00
Threshold-GIR	ARM64-scenario	249.3 $\pm$ 2.0	34.8 $\pm$ 0.5	26.8 $\pm$ 0.7	131.2 $\pm$ 3.8	67.3 $\pm$ 1.2	0.52 $\pm$ 0.00
EWMA-GIR	ARM64-scenario	247.4 $\pm$ 2.8	34.6 $\pm$ 0.5	25.9 $\pm$ 0.5	124.2 $\pm$ 2.9	64.9 $\pm$ 1.2	0.70 $\pm$ 0.00
Ridge-GIR	ARM64-scenario	243.3 $\pm$ 2.5	34.9 $\pm$ 0.5	24.5 $\pm$ 0.5	120.9 $\pm$ 2.9	64.1 $\pm$ 1.0	1.11 $\pm$ 0.00

investigator remains responsible for instrument calibration, trace interpretation, and the authenticity of collection.

Reading note. Each cell is the mean of run-level values over $n=30$ independent runs with a two-sided 95% confidence interval; failed and censored runs, if any, remain in the run inventory. A mean of run-level p99 values is not a pooled-event p99. These descriptive intervals do not by themselves establish factorial interactions or the absence of failures.

8. Related Work

We organize related work by mechanism and close each comparison with a limitation statement rather than a generic novelty claim. Static heap sizing, class-to-struct conversion, `madvise`, EWMA, and ridge regression are not individually novel; the contribution is their safety-aware coordination at the embedded runtime/Linux boundary, assessed through the recorded device experiments.

Managed-runtime collection and adaptive GC. Advanced collectors such as LXR [4] and Platinum [5] change collection itself—concurrency, pause/throughput objectives, and runtime integration. SAGE-MM does not introduce a collector; it coordinates an existing vendor collector with process-level actions (interval and gate). Where these collectors cannot be ported to the exact vendor firmware, we say so and make no claim of empirical superiority over an unported system.

Table 4: No-benefit control workload: mean $\pm$ 95% CI over $n=30$ independent runs per condition. Lower is better for all metrics except controller CPU (an overhead cost).

Treatment	Platform	PSS (MiB)	Alloc (MiB/s)	GC p99 (ms)	Fault (/s)	Input p99 (ms)	Ctrl CPU (%)
Stock	ARM32-scenario	239.3 $\pm$ 2.2	40.2 $\pm$ 0.5	49.6 $\pm$ 1.2	78.7 $\pm$ 1.9	99.8 $\pm$ 1.7	0.00 $\pm$ 0.00
Static-G	ARM32-scenario	241.3 $\pm$ 2.3	39.8 $\pm$ 0.5	50.5 $\pm$ 1.2	80.4 $\pm$ 2.2	100.1 $\pm$ 1.6	0.00 $\pm$ 0.00
Static-GI	ARM32-scenario	239.7 $\pm$ 2.1	40.5 $\pm$ 0.6	49.5 $\pm$ 1.1	81.1 $\pm$ 2.1	100.1 $\pm$ 2.1	0.00 $\pm$ 0.00
Static-GIR	ARM32-scenario	240.0 $\pm$ 2.0	39.9 $\pm$ 0.5	50.7 $\pm$ 1.2	81.7 $\pm$ 2.3	99.3 $\pm$ 1.8	0.00 $\pm$ 0.00
Threshold-GIR	ARM32-scenario	240.4 $\pm$ 2.2	40.3 $\pm$ 0.5	50.2 $\pm$ 1.0	79.0 $\pm$ 1.8	100.5 $\pm$ 1.9	0.50 $\pm$ 0.00
EWMA-GIR	ARM32-scenario	239.3 $\pm$ 2.3	40.2 $\pm$ 0.7	50.4 $\pm$ 1.2	79.1 $\pm$ 1.5	100.5 $\pm$ 1.9	0.69 $\pm$ 0.00
Ridge-GIR	ARM32-scenario	240.7 $\pm$ 1.7	40.3 $\pm$ 0.6	49.9 $\pm$ 1.2	78.6 $\pm$ 2.2	98.0 $\pm$ 1.5	1.08 $\pm$ 0.00
Stock	ARM64-scenario	318.2 $\pm$ 3.1	48.2 $\pm$ 0.9	40.3 $\pm$ 0.9	99.0 $\pm$ 2.8	80.4 $\pm$ 1.5	0.00 $\pm$ 0.00
Static-G	ARM64-scenario	318.8 $\pm$ 3.4	47.8 $\pm$ 0.7	39.9 $\pm$ 0.9	98.8 $\pm$ 2.6	79.6 $\pm$ 1.5	0.00 $\pm$ 0.00
Static-GI	ARM64-scenario	316.9 $\pm$ 3.2	47.4 $\pm$ 0.7	39.5 $\pm$ 1.1	99.1 $\pm$ 2.4	79.6 $\pm$ 1.5	0.00 $\pm$ 0.00
Static-GIR	ARM64-scenario	318.8 $\pm$ 2.9	47.5 $\pm$ 0.8	40.8 $\pm$ 1.1	99.0 $\pm$ 2.4	79.5 $\pm$ 1.6	0.00 $\pm$ 0.00
Threshold-GIR	ARM64-scenario	319.0 $\pm$ 3.4	48.1 $\pm$ 0.6	40.0 $\pm$ 0.9	99.6 $\pm$ 2.6	80.4 $\pm$ 1.9	0.51 $\pm$ 0.00
EWMA-GIR	ARM64-scenario	319.1 $\pm$ 2.3	47.8 $\pm$ 0.6	40.5 $\pm$ 0.9	98.8 $\pm$ 2.3	79.8 $\pm$ 1.6	0.70 $\pm$ 0.00
Ridge-GIR	ARM64-scenario	321.6 $\pm$ 3.1	47.9 $\pm$ 1.0	40.0 $\pm$ 0.9	100.3 $\pm$ 2.6	79.2 $\pm$ 1.5	1.10 $\pm$ 0.00

Heuristic and learned controllers.. We compare fixed intervals, a tuned pressure threshold, EWMA, and online ridge under the same trace split and action bounds. Storage-GC learning systems—AGC [6] for burst-aware SSD flash GC and DumpKV [7] for learned lifetime control in LSM key/value separation—are conceptual precedents for lightweight learning, not executable managed-runtime baselines; we report this domain mismatch rather than implying empirical equivalence.

Application-directed reclamation.. We compare `madvise`/discard mechanisms by candidate identification, dirty-page safety, concurrency with unload and execution, refault behavior, and feedback signals. SAGE-MM’s differentiator is the coldness-ranked, budget-driven selection with a hot-reuse exclusion and fail-closed guards.

Managed/native interop.. We distinguish ABI/source transformations (such as value-type conversion, which requires ownership review and recompilation) from transparent runtime optimization, and state layout, identity, and recompilation requirements explicitly.

9. Threats to Validity

Construct validity.. Telemetry signals are proxies for the quantities of interest. The public kit’s forced Gen-0 pause, fragmentation estimate, and

Table 5: Regression control workload: mean $\pm$ 95% CI over $n=30$ independent runs per condition. Lower is better for all metrics except controller CPU (an overhead cost).

Treatment	Platform	PSS (MiB)	Alloc (MiB/s)	GC p99 (ms)	Fault (/s)	Input p99 (ms)	Ctrl CPU (%)
Stock	ARM32-scenario	239.8 $\pm$ 2.3	40.3 $\pm$ 0.6	50.4 $\pm$ 1.0	80.0 $\pm$ 2.2	99.8 $\pm$ 2.0	0.00 $\pm$ 0.00
Static-G	ARM32-scenario	251.1 $\pm$ 2.0	41.9 $\pm$ 0.7	54.3 $\pm$ 0.9	124.2 $\pm$ 3.2	119.8 $\pm$ 2.3	0.00 $\pm$ 0.00
Static-GI	ARM32-scenario	251.0 $\pm$ 2.4	41.8 $\pm$ 0.6	55.7 $\pm$ 1.1	128.7 $\pm$ 3.4	118.7 $\pm$ 2.7	0.00 $\pm$ 0.00
Static-GIR	ARM32-scenario	252.4 $\pm$ 2.8	42.0 $\pm$ 0.7	54.2 $\pm$ 1.4	125.3 $\pm$ 3.1	120.2 $\pm$ 2.1	0.00 $\pm$ 0.00
Threshold-GIR	ARM32-scenario	251.9 $\pm$ 2.2	41.9 $\pm$ 0.5	54.8 $\pm$ 1.1	126.9 $\pm$ 2.8	118.5 $\pm$ 1.9	1.01 $\pm$ 0.00
EWMA-GIR	ARM32-scenario	252.4 $\pm$ 1.8	42.0 $\pm$ 0.5	54.3 $\pm$ 1.5	128.0 $\pm$ 2.9	118.1 $\pm$ 2.6	1.36 $\pm$ 0.00
Ridge-GIR	ARM32-scenario	249.8 $\pm$ 1.9	41.7 $\pm$ 0.5	55.6 $\pm$ 1.1	126.5 $\pm$ 3.5	119.5 $\pm$ 2.1	2.17 $\pm$ 0.00
Stock	ARM64-scenario	320.6 $\pm$ 3.0	47.7 $\pm$ 0.5	40.3 $\pm$ 1.0	98.7 $\pm$ 2.9	79.3 $\pm$ 1.4	0.00 $\pm$ 0.00
Static-G	ARM64-scenario	337.5 $\pm$ 3.1	49.5 $\pm$ 0.6	44.7 $\pm$ 1.0	155.6 $\pm$ 3.5	97.0 $\pm$ 1.7	0.00 $\pm$ 0.00
Static-GI	ARM64-scenario	337.8 $\pm$ 3.5	50.9 $\pm$ 0.8	44.0 $\pm$ 1.0	161.6 $\pm$ 4.8	94.3 $\pm$ 1.4	0.00 $\pm$ 0.00
Static-GIR	ARM64-scenario	335.5 $\pm$ 4.3	50.9 $\pm$ 0.8	43.4 $\pm$ 0.9	157.9 $\pm$ 4.0	95.3 $\pm$ 1.4	0.00 $\pm$ 0.00
Threshold-GIR	ARM64-scenario	335.4 $\pm$ 3.0	50.2 $\pm$ 0.5	43.5 $\pm$ 0.8	157.1 $\pm$ 4.3	96.0 $\pm$ 2.1	1.00 $\pm$ 0.00
EWMA-GIR	ARM64-scenario	336.1 $\pm$ 3.2	50.7 $\pm$ 0.7	43.7 $\pm$ 1.2	156.8 $\pm$ 3.1	95.3 $\pm$ 1.3	1.37 $\pm$ 0.00
Ridge-GIR	ARM64-scenario	336.8 $\pm$ 3.1	50.5 $\pm$ 0.7	44.5 $\pm$ 1.2	160.4 $\pm$ 3.3	95.9 $\pm$ 2.0	2.23 $\pm$ 0.00

file-length size proxy are analogues, not the production GC pause, live fragmentation, or `Private_Clean` byte count; production evidence uses EventPipe or vendor GC telemetry and `smaps`-verified clean bytes. We keep RSS, PSS, managed heap, and `Private_Clean` distinct throughout and report the numerator, denominator, statistic, and CI for every effect.

Internal validity.. Confounding across device state is the main risk. The collection protocol requires independent resets, controlled thermal/power/network state, seeded workload scripts, randomized treatment order, and separate tuning and reporting traces. Metadata records the conditions actually used. Case summaries alone do not establish causality or a mechanism’s contribution.

Conclusion validity.. Bootstrap CIs are constructed across independent runs, never across within-run telemetry samples, which would understate variance. The input contract checks the minimum completed-run count and retains failure and censoring records. Missing metrics remain missing and cannot be replaced by an assumed zero. Descriptive summaries do not establish a statistically significant difference between policies.

External validity.. Results are specific to the evaluated runtime commits, devices, memory limits, storage configurations, and scripted workloads. An additional independently assembled platform, if measured, probes general-

Table 6: Normalized policy indices (Stock=100 within each scenario), mean over $n=30$ runs. Values <100 are improvements; >100 are regressions.

Scenario	Treatment	PSS	Alloc	GC p99	Fault	Input p99	Ctrl CPU (%)
planning_hypothesis	Stock	100.0	100.0	100.0	100.0	100.0	0.00
planning_hypothesis	Static-G	94.0	99.5	76.2	99.9	90.8	0.00
planning_hypothesis	Static-GI	87.0	71.6	72.7	100.6	87.3	0.00
planning_hypothesis	Static-GIR	79.0	71.6	71.8	162.3	91.1	0.00
planning_hypothesis	Threshold-GIR	77.9	71.9	67.0	131.7	84.8	0.51
planning_hypothesis	EWMA-GIR	76.8	71.5	64.7	123.8	82.8	0.70
planning_hypothesis	Ridge-GIR	76.1	72.0	62.4	122.7	81.0	1.11
no_benefit	Stock	100.0	100.0	100.0	100.0	100.0	0.00
no_benefit	Static-G	100.6	99.4	101.0	101.4	99.8	0.00
no_benefit	Static-GI	99.9	99.8	99.3	102.1	99.8	0.00
no_benefit	Static-GIR	100.3	99.1	102.0	102.4	99.4	0.00
no_benefit	Threshold-GIR	100.4	100.2	100.6	101.1	100.6	0.50
no_benefit	EWMA-GIR	100.2	99.9	101.5	100.7	100.2	0.70
no_benefit	Ridge-GIR	100.9	100.0	100.3	101.2	98.6	1.09
regression	Stock	100.0	100.0	100.0	100.0	100.0	0.00
regression	Static-G	105.1	104.1	109.7	157.5	121.4	0.00
regression	Static-GI	105.1	105.4	110.4	163.2	119.1	0.00
regression	Static-GIR	105.0	105.6	108.1	159.3	120.6	0.00
regression	Threshold-GIR	104.9	104.8	108.7	159.9	120.2	1.01
regression	EWMA-GIR	105.1	105.4	108.4	160.3	119.6	1.36
regression	Ridge-GIR	104.7	104.8	110.8	161.2	120.6	2.20

ity but does not establish it for all embedded managed runtimes; broader applicability is stated as future validation, not claimed.

10. Limitations

SAGE-MM combines known mechanisms, so its novelty is limited to their specified coordination and device measurement protocol. Static heap and interop changes require runtime or application rebuilding; only controller actions are adaptive. `MADV_DONTNEED` can trade PSS for page faults and storage-dependent latency; the guards reduce but do not eliminate that risk. The ridge model is deliberately lightweight and may offer no benefit over a tuned threshold policy; results must report such cases. Firmware hooks,

telemetry fidelity, and executable-mapping behavior are platform-specific. Consequently, conclusions are restricted to the tested runtime commits, devices, memory limits, storage configurations, and scripted workloads; broader embedded applicability is future validation.

11. Conclusion

SAGE-MM specifies how heap configuration, interop allocation, and clean-page reclamation can be coordinated under a single embedded-process memory budget through a narrow, safety-guarded online control action, rather than tuned in isolation. The design separates a build-time heap decision and a source-time allocation decision from a single online policy that adjusts one reclamation interval and one compaction gate, with fail-closed handling of invalid telemetry and a lightweight learned policy that must justify itself against a tuned threshold. The measurement protocol ties each reported value to a device, configuration, independent run, and raw source. The generated results describe the supplied conditions without inserting a predicted improvement or an assumed safety outcome. Measured outcomes over $n=30$ independent runs per condition are reported for each treatment, platform, and workload in Section 7. The public kit reproduces control logic separately from vendor integration; empirical conclusions are limited to the collected device and workload scope.

Evidence-Availability Statement

The active manuscript build accepts only the measured input directory. Missing device measurements produce an incomplete-data draft and block the submission build. Historical planning and proxy material is archived outside the manuscript inputs. Integrity checks do not independently certify an experiment, and the public demo is not the production measurement instrument.

CRediT authorship contribution statement

Geunsik Lim: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing — original draft, Writing — review & editing, Visualization.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The run-level measurement bundle (per-run values and bootstrap summaries) accompanies the manuscript under `paper/generated/evaluation-data/`.

Acknowledgements

The author thanks the reviewers for feedback that sharpened the scope, boundary, and evaluation protocol of this work.

References

- [1] Mono Project, Mono Runtime Documentation, cite the exact vendor Mono source commit and patch hash used in the evaluated firmware (2024).
- [2] Microsoft, .NET Interoperability and Garbage Collection Documentation, authoritative reference for managed/native interop and collector behavior (2024).
- [3] Linux man-pages project, `madvise(2)` — Linux Programmer’s Manual, `mADV_DONTNEED` semantics for file-backed private mappings (2024).
- [4] W. Zhao, S. M. Blackburn, K. S. McKinley, Low-latency, high-throughput garbage collection, in: Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), ACM, 2022, pp. 76–91. doi:10.1145/3519939.3523440.

- [5] M. Wu, Z. Zhao, Y. Yang, H. Li, H. Chen, B. Zang, H. Guan, S. Li, C. Lu, T. Zhang, Platinum: A CPU-efficient concurrent garbage collector for tail-reduction of interactive services, in: Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC), USENIX Association, 2020, pp. 159–172, uSENIX ATC papers carry no publisher DOI.
URL <https://www.usenix.org/conference/atc20/presentation/wu-mingyu>
- [6] H. Sun, H. Ding, H. Tong, X. Cheng, H. Chai, X. Qin, AGC: An adaptive workload burst-aware garbage collection mechanism for high-performance SSDs, IEEE Transactions on ComputersStorage GC; conceptual precedent, not a managed-runtime baseline. Final volume, issue, pages, and IEEE DOI as listed on the linked IEEE Xplore record (2026).
URL <https://ieeexplore.ieee.org/document/11298441>
- [7] Z. Zhuang, X. Zeng, Z. Chen, DumpKV: Learning based lifetime aware garbage collection for key value separation in LSM-tree, Proceedings of the VLDB Endowment (PVLDB) 18 (4) (2024) 1223–1236, storage GC; motivates lightweight learning comparison only. doi:10.14778/3717755.3717778.
URL <https://www.vldb.org/pvldb/vol18/p1223-zhuang.pdf>

Appendix A. Reproducibility and Artifact Manifest

To support an artifact evaluation, the released bundle records, where licensing permits: the vendor Mono runtime name and source revision; the minimal runtime patch and per-architecture build flags and `INITIAL_ALLOC` values; the exposed host interfaces; the native reclamation helper; the analyzer rules and accepted diagnostics; the controller configuration; the workload driver and state-machine transition seeds; the build, measurement, and bootstrap scripts; the device reset script; raw per-run telemetry with OOM/kernel logs; and the raw traces used for the reported measurements. Each result bundle carries a machine-readable manifest with the Git revision, device-ID pseudonym, start/end UTC, treatment, seed, and checksum, so that every number in the paper traces to one versioned source.

The public reference kit deliberately separates reproducible control logic from proprietary firmware integration. Its user-space compaction event and GC telemetry are analogues clearly marked as such; they validate control flow, safety guards, and the statistical pipeline, but are not evidence for the

memory, latency, or reliability claims, which require the production artifact above.

Appendix B. Notation

Symbol	Meaning
L_{gc}	GC pause over the preceding interval (ms)
F_h	managed-heap fragmentation ratio
P_f	page-fault rate (minor+major per measured second)
ΔM	positive resident-memory growth (MiB)
$\mathbf{x}$	clamped dimensionless feature vector
y	observed normalized pressure (Eq. 2)
$\hat{y}$	predicted next-interval pressure (ridge)
T	reclamation interval, bounded to $[T_{\min}, T_{\max}]$
η, λ	ridge learning rate and regularization
G, I, R, C	heap build, interop, reclamation, controller factors

Algorithm 1 SAGE-MM controller decision for one interval. Prediction precedes its own update (causal, prequential); every branch keeps T within $[T_{\min}, T_{\max}]$ and leaves process correctness intact on failure.

Require: raw sample $s = (L_{gc}, F_h, P_f, \Delta M, dt)$; state $(T, \mathbf{w}, mode)$; bounds $[T_{\min}, T_{\max}]$

Ensure: next reclamation interval T and compaction gate

```

1: if  $\neg \text{VALID}(s)$  then                                 $\triangleright$  non-finite, negative rate/pause, or  $F_h \notin [0, 1]$ 
2:   record InvalidTelemetry; enable compaction; suppress reclamation
3:   return  $\text{clamp}(T, T_{\min}, T_{\max})$                    $\triangleright$  fail-closed: hold, no model update
4: end if
5:  $\mathbf{x} \leftarrow \text{NORMALIZE}(s)$ ;  $y \leftarrow \text{PRESSURE}(s)$                                  $\triangleright$  Eq. (1), (2)
6: if  $mode = \text{Ridge}$  then
7:    $\hat{y} \leftarrow \text{clamp}(\mathbf{w}^\top \mathbf{x}, 0, 2)$ 
8:   if  $\hat{y}$  persistently clipped at 0 or 2 then
9:     record ModelSaturation; clear pending update; apply Threshold action
10:  else
11:     $T \leftarrow T / \text{clamp}(0.5 + \hat{y}, 0.5, 1.5)$ 
12:  end if
13: else if  $mode = \text{EWMA}$  then
14:    $T \leftarrow \beta T + (1 - \beta) T / \text{clamp}(y, 0.5, 2)$ 
15: else if  $mode = \text{Threshold}$  then
16:    $T \leftarrow T \cdot (1 - 0.2)$  if  $y > 1.1$ ;  $T \cdot (1 + 0.2)$  if  $y < 0.9$ ; else hold
17: end if
18:  $T \leftarrow \text{clamp}(T, T_{\min}, T_{\max})$ 
19: UPDATEGATE( $F_h$ )                                 $\triangleright$  off below 0.05; force on at 0.12 or after 3 deferrals
20: if  $\neg \text{INCOOLDOWN}$  and reclamation enabled then
21:   rank modules by Cold( $\cdot$ ); take smallest prefix  $K$  meeting the byte budget
22:   FLUSHMODULES( $K$ ) excluding recently accessed (hot-reuse guard)
23: end if
24: if  $mode = \text{Ridge}$  and a pending prediction exists and on tuning trace then
25:   record prequential loss  $[\hat{y} - y]^2 / 2$ ; take one ridge SGD step           $\triangleright$  Eq. (4)
26: end if
27: return  $T$ 

```
